

APPLICATION: AI-BASED CHATBOT PROCESSING

1. Aim

To determine the time complexity of the recurrence relation using the Master Theorem and analyze the performance of an AI-based chatbot processing system.

2. Problem Statement

An AI chatbot processes user queries using the recurrence relation:

$$T(n) = T(n/2) + \log n$$

Apply the Master Theorem to determine the time complexity. Identify the parameters and justify the selected case. Discuss its impact on response time as user queries increase.

3. Theory

The Master Theorem is used to solve recurrence relations of the form:

$$T(n) = aT(n/b) + f(n)$$

Where:

- **a** = Number of recursive subproblems
- **b** = Factor by which the problem size is reduced
- **f(n)** = Cost of dividing and combining the subproblems

4. Master Theorem Analysis

Given,

$$T(n) = T(n/2) + \log n$$

Parameters

- **a = 1**
- **b = 2**

- $f(n) = \log n$

Calculate,

$$n^{\log_b a}$$

$$n^{\log_2 1} = n^0 = 1$$

Since

$$f(n) = \Theta(n^0 \log n)$$

Master Theorem **Case 2** is applicable.

Therefore,

$$T(n) = \Theta((\log n)^2)$$

5. Time Complexity

Time Complexity

$$\Theta((\log n)^2)$$

The chatbot processes user queries efficiently because the problem size is reduced by half at every recursive step. The response time increases very slowly even as the number of user queries becomes very large.

6. Space Complexity

Each recursive call reduces the input size by half.

Recursion depth:

$$\log_2 n$$

Hence,

$$\text{Space Complexity} = \Theta(\log n)$$

7. Performance Analysis

- The chatbot divides each query into smaller subproblems.
- Only logarithmic additional work is performed at every recursive step.
- The response time grows slowly with increasing user requests.
- This makes the chatbot suitable for handling a large number of users efficiently.

8. Algorithm

1. Identify the values of **a**, **b**, and **f(n)**.
2. Compute $n^{\log_b a}$.
3. Compare $f(n)$ with $n^{\log_b a}$.
4. Select the appropriate Master Theorem case.
5. Determine the time and space complexity.

9. Source Code

```
#include <stdio.h>

int main()
{
    printf("=====\n");
    printf(" AI-BASED CHATBOT PROCESSING\n");
    printf("=====\n\n");

    printf("Recurrence Relation:\n");
    printf("T(n) = T(n/2) + log n\n\n");

    printf("Master Theorem Parameters:\n");
    printf("a = 1\n");
    printf("b = 2\n");
    printf("f(n) = log n\n\n");

    printf("Step 1:\n");
    printf("n^(log2 1) = n^0 = 1\n\n");

    printf("Step 2:\n");
    printf("f(n) = log n = Theta(n^0 log n)\n\n");

    printf("Step 3:\n");
    printf("Master Theorem Case 2 is applicable.\n\n");

    printf("Result:\n");
    printf("Time Complexity : Theta((log n)^2)\n");
    printf("Space Complexity : Theta(log n)\n\n");

    printf("Performance Analysis:\n");
    printf("The chatbot divides user queries into smaller subproblems.\n");
    printf("Response time grows very slowly as the number of queries increases,\n");
    printf("making the chatbot efficient for large-scale applications.\n");
    printf("\nPress Enter to exit...");
    getchar();
    return 0;
}
```

10. Output

```
=====
AI-BASED CHATBOT PROCESSING
=====

Recurrence Relation:
T(n) = T(n/2) + log n

Master Theorem Parameters:
a = 1
b = 2
f(n) = log n

Step 1:
n^(log2 1) = n^0 = 1

Step 2:
f(n) = log n = Theta(n^0 log n)

Step 3:
Master Theorem Case 2 is applicable.

Result:
Time Complexity : Theta((log n)^2)
Space Complexity : Theta(log n)

Performance Analysis:
The chatbot divides user queries into smaller subproblems.
Response time grows very slowly as the number of queries increases,
making the chatbot efficient for large-scale applications.
```

11. Result

Master Theorem **Case 2** is applicable.

Time Complexity

$$\Theta((\log n)^2)$$

Space Complexity

$$\Theta(\log n)$$

12. Conclusion

The AI-Based Chatbot Processing System follows the recurrence relation $T(n) = T(n/2) + \log n$. Applying the Master Theorem shows that the algorithm has **Time Complexity** $\Theta((\log n)^2)$ and **Space Complexity** $\Theta(\log n)$. Therefore, the chatbot scales efficiently and provides fast responses even for a large number of user queries.

